ChatGPT

Secure Agentic Browser Extension – 6-Phase Development Plan

Installment 1 – Chrome Extension Foundation

Begin by **building atop Nanobrowser's Chrome extension framework**. Nanobrowser is officially a Chrome (and Edge) extension platform [1], so use Manifest V3 with minimal permissions (follow the principle of least privilege) [2]. Define your extension manifest to request only needed host and tab permissions and specify a strict Content Security Policy. Set up the UI (toolbar popup, sidebar) and background scripts for Nanobrowser's multi-agent workflow. Include a "risk assessment" component early: for example, a background page that inspects each loaded URL and content. Perform an initial **security risk assessment** of your extension itself (static analysis, OWASP scanning, Snyk) as recommended [2]. In code, follow Chrome security best practices: validate/sanitize all inputs, avoid `eval` and inline scripts, and use Chrome's storage APIs with encryption for any sensitive data [2] [3]. In summary, phase 1 establishes the Chrome-extension skeleton (manifest, sandboxing, Nanobrowser integration) under strict security settings [2] [1].

- Leverage Nanobrowser's architecture (multi-agent, local execution) but tailor it to your security needs.
- Use manifest V3, restrict permissions, enforce strict CSP – as recommended for safe extensions [2] [1].
- Implement a background risk-checker that intercepts page loads and scans content for phishing/risk signals.
- Incorporate extension-specific security audits and code reviews early.

Installment 2 – Local-First Data Processing & Federated Learning

Design all scanning and inference to happen **locally in the browser**. Nanobrowser's philosophy is "everything runs in your local browser" [4], so keep user data and models on-device. Use **browser-based ML frameworks** (e.g. TensorFlow.js or ONNX Runtime) to run inference in JavaScript/WebAssembly. For example, ONNX Runtime can run models in-browser using WebAssembly and even WebGPU for GPU acceleration [5]. This lets the extension train or update models directly on the client without sending raw data to a server. Architect your system for **federated learning**: each user's extension trains on their own interaction data, then only model updates (gradients) are sent to a central aggregator. Federated learning "allows training models without transfer of sensitive data" by keeping data on-device [6]. To ensure privacy compliance (GDPR, etc.), combine FL with techniques like secure aggregation or differential privacy on the updates [7]. In practice: set up a lightweight local database (IndexedDB) to store feature vectors or examples of suspicious page patterns. Periodically, bundle model updates to send to your server (or a peer-to-peer system) for federated averaging.

- Execute all detection algorithms (risk assessment, phishing classification) locally using WebAssembly/JavaScript ML libraries [5].
- Use ONNX Runtime or TensorFlow.js to load your trained models and run inference on-page content.

- Implement federated averaging: each client computes gradients on its local data and sends *only* model deltas back, preserving raw data privacy [6].
- Ensure encryption or secure aggregation of updates, since FL alone isn't enough for full privacy compliance [7].

Installment 3 – On-Screen Content Scanning & Phishing Detection

Build a **content analysis engine** that inspects anything visible in the browser. Use content scripts to traverse the DOM of every page: extract URLs, form fields, images, and text. For images or complex layouts, you can even capture the visible page as an image for analysis via the Chrome API (e.g. `chrome.tabs.captureVisibleTab()` with `<all_urls>` or `activeTab` permission [8]). Once you have page data (text or screenshot), run your phishing/risk detection model. For example, you might OCR an image or scan the DOM for known phishing markers (fake login forms, mismatched links, hidden iframes, etc.). Use your trained ML classifier on the extracted content: many academic systems use public datasets (e.g. PhishTank) and features like URL length, domain age, SSL usage, and HTML cues to distinguish phishing [9]. (NoPhish, for instance, used 22 features from PhishTank and found Random Forest gave high precision [9].) In code, whenever a new page loads or changes, trigger the scanner. If a page scores high risk, immediately alert the user or block the content. Log any warnings into your secure log (see next installment). Over time, the agents in Nanobrowser can also query the user ("Is this site safe?") and feed responses back to your federated learner.

- Inject content scripts on all pages to parse the DOM and extract features (URLs, text snippets, form elements, images).
- Optionally use `chrome.tabs.captureVisibleTab()` to snapshot the page and run image analysis/OCR [8].
- Run a phishing-detection ML model (trained offline) on the extracted data. For example, use features like those in the PhishTank dataset and a classifier (Random Forest was effective [9]).
- If a page is classified as high-risk or phishing, show a warning/overlay and prevent further navigation. Record this event in your log.
- Continuously refine the detector by feeding new flagged cases into your federated training pipeline.

Installment 4 – Tamper-Evident Logging (Blockchain Option)

Implement an **append-only audit log** for all agent activities (e.g. detections, user responses, decisions). Each log entry should include a timestamp, action/event type, and a cryptographic hash of the *previous* entry. This creates a hash chain: altering any past log invalidates all subsequent hashes [10]. Use SHA-256 or a similar hash via the Web Crypto API to compute `entry_hash = H(timestamp || action || data || previous_hash)`. Store the log in browser storage (IndexedDB or browser files). To make logs *tamper-evident*, periodically "anchor" your log's Merkle root or latest hash in a secure external system. For example, you could publish a daily Merkle root on a public blockchain or a trusted timestamping service [11]. This way, even if a user tries to modify their local log, they cannot change the committed root. If blockchain is cost-prohibitive, consider permissioned ledgers (Hyperledger) or even a Git-backed append-only server. The key is that any attempt to rewrite history is detectable.

- For each logged event (e.g. "Phishing alert on URL X"), compute a new hash that includes the previous entry's hash [10]. Store entries sequentially.
- Use Merkle trees for scalability if logs grow large: e.g. batch entries into daily blocks and compute a Merkle root per block.

- Anchor each block's root in a trustable way. A cost-effective approach: periodically commit the hash to a public ledger or blockchain transaction [11].
- In code, implement a verification routine that recomputes hashes to ensure no tampering: if recomputed root ≠ anchored root, raise an alert.
- This tamper-evident log ensures a forensic trail of the extension's actions and helps with compliance/auditing.

Installment 5 – Core Development (Models & Code from Scratch)

Develop your models and codebase **from the ground up** (do not rely on opaque third-party agents). First, gather training data: e.g., compile a set of legitimate and phishing page samples (use PhishTank, Spam Assassin URLs, etc.). Extract features or label screenshots as needed. Train a classification model in Python or JS (examples: Random Forest, SVM, or a light neural net). Keep the model small enough to run fast in-browser (e.g. via TensorFlow.js or ONNX). After training, export the model to ONNX or TensorFlow format and load it into the extension. Simultaneously, write robust code modules for content scanning, state management, and UI. Follow secure-coding standards at all times: sanitize any input you process, avoid dynamic code execution (`eval`), and enforce content scripts only run on HTTPS pages [3]. Use version control and code reviews to eliminate bugs. Build unit tests for each component (detection logic, logging, UI alerts). Iteratively simulate attacks (e.g. known phishing pages) to refine your model and rules. By the end of this phase, you should have a fully functioning extension with custom-built phishing/risk detectors and security features (logging, sandboxing) all integrated.

- **Train your own classifiers:** Use open datasets (PhishTank, Alexa top sites, etc.) and standard ML algorithms. For example, [NoPhish] trained random forests on PhishTank features [9].
- **Export to browser:** Convert models to ONNX or TensorFlow.js format for in-browser inference. Leverage the browser's JavaScript ML runtime [5].
- **Secure code practices:** Rigorously validate/sanitize all inputs and avoid unsafe constructs (as per Chrome security guidelines [3]). Ensure your background scripts and content scripts communicate safely.
- **Testing and iteration:** Unit-test detection, risk scoring, and UI flows. Simulate malicious pages to tune thresholds. Document your code and maintain a changelog for audits.

Installment 6 – Mac (M1/M4) Compatibility & Performance

Finally, ensure the extension runs smoothly on Apple Silicon (M1/M4) Macs. Fortunately, Chrome extensions run natively on all architectures, but pay attention to any native dependencies in your toolchain. Use cross-platform libraries: WebAssembly-based ML (TensorFlow.js, ONNX) runs on ARM just as on x86 [5]. If you use ONNX Runtime with WebGPU, it will leverage the Mac's GPU for acceleration [5]. Build and test on a Mac environment: for example, install Chrome on an M1/M4 Mac and load your unpacked extension. Profile performance: ensure that your content scanning and ML inference are efficient (WebGPU/WebGL can help). Avoid node modules or binaries without ARM support. In short, keep the extension's logic in pure JS/WebAssembly so it's architecture-agnostic. Test all features (page scan, logging, federated update) on the Mac to catch any platform-specific issues.

- **Use cross-platform frameworks:** TensorFlow.js and ONNX Runtime use WebAssembly/WebGPU, which run on Apple Silicon [5]. No native compilation is needed on the client.
- **Optimize for ARM:** If using Node tools (for build or training), install Apple Silicon-compatible versions or use Rosetta 2 for any legacy code.
- **Leverage GPU:** ONNX + WebGPU can utilize the Mac's GPU for faster inference [5]. Enable WebGPU in Chrome if needed.

- **Extensive testing:** Run the extension on M1/M4 Macs and on Intel to ensure parity. Monitor CPU/GPU load when scanning pages. Fix any memory or speed issues.

By following these six phases, you will build a robust Chrome extension atop Nanobrowser with on-device phishing/risk detection, secure logging, and continuous improvement via federated learning. Each step embeds security by design [2] [3] and leverages state-of-the-art techniques (local ML [5], tamper-proof logs [10] [11], privacy-preserving learning [6]) to meet the project goals.

Sources: Nanobrowser documentation [1] [4]; Chrome extension security best practices [2] [3]; NoPhish phishing extension case study [9]; federated phishing ML research [6]; browser ML frameworks (ONNX/WebGPU) [5]; tamper-evident logging design [10] [11]; Chrome `captureVisibleTab()` API [8].

---

[1] [4] GitHub - nanobrowser/nanobrowser: Open-Source Chrome extension for AI-powered web automation. Run multi-agent workflows using your own LLM API key. Alternative to OpenAI Operator.
https://github.com/nanobrowser/nanobrowser

[2] [3] Best Practices for Secure Chrome Extension Development
https://www.creolestudios.com/chrome-extension-development-best-practices-for-security/

[5] InFL-UX: A Toolkit for Web-Based Interactive Federated Learning
https://arxiv.org/html/2503.04318v1

[6] [7] City Research Online - Privacy-Preserving Federated Learning for Phishing Detection
https://openaccess.city.ac.uk/id/eprint/35006/

[8] tabs.captureVisibleTab() - Mozilla | MDN
https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/tabs/captureVisibleTab

[9] NoPhish: Efficient Chrome Extension for Phishing Detection Using Machine Learning Techniques
https://aisecurity-portal.org/en/literature-database/nophish-efficient-chrome-extension-for-phishing-detection-using-machine-learning-techniques/

[10] [11] How do you design tamper-evident audit logs (Merkle trees, hashing)?
https://www.designgurus.io/answers/detail/how-do-you-design-tamperevident-audit-logs-merkle-trees-hashing